LangChain 1.0 全面学习资料

一、LangChain 1.0 概述

1.1 什么是LangChain

LangChain 是一个开源的大语言模型（LLM）应用开发框架，旨在简化基于LLM的复杂应用构建流程。其核心思想是“链接”（Chain）各类组件，包括LLM模型、数据存储、外部API、工具等，让开发者能够快速搭建具备上下文理解、多步推理、外部交互能力的LLM应用。

LangChain 1.0 是其重要的里程碑版本，相比早期版本，在架构设计、组件稳定性、功能完整性、性能优化等方面均有显著提升，正式确立了“组件化+可组合”的核心架构，为企业级LLM应用开发提供了更可靠的支撑。

1.2 LangChain 1.0 的核心价值

- **组件化开发**：将LLM应用的核心模块（如提示词管理、模型调用、记忆存储、工具集成等）封装为独立组件，开发者可按需组合，降低重复开发成本。
- **上下文感知能力**：通过“记忆”（Memory）组件高效管理对话历史、任务状态等上下文信息，让LLM能够基于历史交互进行连续推理。
- **外部能力扩展**：支持集成各类外部工具（搜索引擎、数据库、代码执行器、第三方API等），突破LLM自身的知识局限和功能边界。
- **多模态支持**：1.0版本强化了对文本、图像、音频等多模态数据的处理能力，适配多模态LLM模型的开发需求。
- **企业级特性**：提供完善的错误处理、日志记录、性能监控、安全性优化等企业级功能，适配生产环境部署。

1.3 LangChain 1.0 的适用场景

- 对话机器人（客服机器人、私人助手、多轮对话系统）
- 文档问答系统（PDF/Word/网页等文档的智能检索与问答）
- 代码生成与调试（自动化代码编写、错误修复、代码解释）
- 数据分析与可视化（连接数据库进行查询、数据处理、生成可视化报告）
- 智能代理（自动规划任务、调用工具完成复杂目标，如旅行规划、学术调研）
- 多模态应用（图像描述生成、语音转文字+智能回复、视频内容分析）

二、LangChain 1.0 环境搭建与基础配置

2.1 环境依赖

LangChain 1.0 基于Python开发，核心依赖如下：

- Python 版本：3.8+（推荐3.10+，兼容性更好）
- 核心依赖库：langchain（1.0+版本）、openai/anthropic/google-generativeai（对应LLM模型的SDK）、numpy、pandas（数据处理）、requests（API调用）
- 可选依赖库：chromadb/pinecone（向量数据库）、pdfplumber/python-docx（文档解析）、matplotlib/seaborn（数据可视化）

2.2 安装步骤

基础安装（核心框架）

Code block

```
pip install langchain==1.0.0  # 指定1.0.0版本，避免版本兼容问题
```

安装LLM模型依赖（以OpenAI为例）

Code block

```
pip install openai  # OpenAI模型SDK
pip install anthropic  # Claude模型SDK（可选）
pip install google-generativeai  # 谷歌Gemini模型SDK（可选）
```

安装可选组件依赖（文档处理、向量数据库等）

Code block

```
# 文档解析依赖
pip install pdfplumber python-docx python-multipart

# 向量数据库依赖
pip install chromadb pinecone-client weaviate-client

# 数据可视化依赖
pip install matplotlib seaborn
```

2.3 基础配置（以OpenAI模型为例）

使用LangChain调用外部LLM模型时，需要配置对应的API密钥。常见配置方式有两种：

方式1：直接在代码中配置

Code block

```
from langchain_openai import OpenAI

# 初始化OpenAI模型（LangChain 1.0推荐使用langchain_openai子模块）
llm = OpenAI(
    api_key="your-openai-api-key",  # 替换为你的OpenAI API密钥
    model_name="gpt-3.5-turbo-instruct",  # 模型名称
    temperature=0.7  # 随机性参数（0-1，值越高越随机）
)
```

方式2：通过环境变量配置（推荐，更安全）

1. 设置环境变量（Windows/Mac/Linux通用方式）：

Code block

```
# Windows（命令行）
set OPENAI_API_KEY=your-openai-api-key

# Mac/Linux（终端）
export OPENAI_API_KEY=your-openai-api-key
```

2. 代码中自动读取环境变量：

Code block

```
from langchain_openai import OpenAI
import os

# 自动从环境变量读取API密钥
llm = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY"),
    model_name="gpt-3.5-turbo-instruct",
    temperature=0.7
)
```

三、LangChain 1.0 核心组件详解

LangChain 1.0 的核心架构围绕“组件化”设计，主要包含6大核心组件：**模型（Models）**、**提示词（Prompts）**、**链（Chains）**、**记忆（Memory）**、**工具（Tools）**、**代理（Agents）**。各组件可独

立使用，也可灵活组合实现复杂功能。

3.1 模型（Models）：LLM能力的核心入口

Models 组件是LangChain与各类LLM模型交互的统一接口，支持主流的闭源模型（OpenAI、Claude、Gemini）和开源模型（Llama 2、Mistral、Qwen）。LangChain 1.0 对Models组件进行了重构，提供了更统一的调用方式和更丰富的模型适配。

3.1.1 模型类型

- **文本生成模型（LLMs）**：专注于文本生成，如GPT-3.5、GPT-4、Claude 3、Llama 2等。
- **聊天模型（Chat Models）**：专门用于对话场景，基于消息（Message）格式交互，如GPT-3.5-turbo、GPT-4-turbo、Claude 3 Chat等。
- **多模态模型（Multi-modal Models）**：支持文本、图像、音频等多模态输入输出，如GPT-4o、Gemini Pro Vision、Claude 3 Opus（支持图像）。
- **嵌入模型（Embedding Models）**：将文本转换为向量表示，用于语义检索、相似度计算，如text-embedding-3-small、all-MiniLM-L6-v2。

3.1.2 核心使用示例

1. 文本生成模型（LLM）调用

Code block

```
from langchain_openai import OpenAI

# 初始化文本生成模型
llm = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY"),
    model_name="gpt-3.5-turbo-instruct",
    temperature=0.5,
    max_tokens=200  # 最大生成 tokens 数
)

# 简单文本生成
response = llm.invoke("解释一下LangChain 1.0的核心优势")
print(response)
```

2. 聊天模型（Chat Model）调用

Code block

```
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage, SystemMessage

```

```
# 初始化聊天模型
chat_llm = ChatOpenAI(
    api_key=os.getenv("OPENAI_API_KEY"),
    model_name="gpt-3.5-turbo",
    temperature=0.3
)

# 构建对话消息（SystemMessage：系统提示，HumanMessage：用户输入）
messages = [
    SystemMessage(content="你是一个LangChain技术顾问，负责解答用户关于LangChain 1.0的问题"),
    HumanMessage(content="LangChain 1.0中的Chat Model和LLM有什么区别？")
]

# 调用聊天模型
response = chat_llm.invoke(messages)
print(response.content)
```

3. 嵌入模型（Embedding Model）调用

Code block

```
from langchain_openai import OpenAIEmbeddings

# 初始化嵌入模型
embeddings = OpenAIEmbeddings(
    api_key=os.getenv("OPENAI_API_KEY"),
    model="text-embedding-3-small"
)

# 生成文本嵌入向量
text = "LangChain 1.0 核心组件包括Models、Prompts、Chains等"
embedding_vector = embeddings.embed_query(text)  # 用于查询（生成单个向量）
# embedding_vectors = embeddings.embed_documents([text1, text2])  # 用于文档批量嵌入

print(f"嵌入向量维度：{len(embedding_vector)}")
print(f"嵌入向量前5个值：{embedding_vector[:5]}")
```

3.2 提示词（Prompts）：控制LLM输出的关键

Prompts 组件用于管理和优化提示词模板，帮助开发者更高效地构建结构化提示词，提升LLM输出的准确性和一致性。LangChain 1.0 强化了提示词的动态填充、格式校验、多模态提示支持等功能。

3.2.1 核心概念

- **提示词模板（Prompt Template）**：定义固定格式的提示词框架，包含可动态替换的变量，用于批量生成个性化提示词。
- **聊天提示词模板（Chat Prompt Template）**：专门用于聊天模型的提示词模板，基于消息列表（System/Human/AIMessage）构建。
- **输出解析器（Output Parser）**：将LLM的原始输出转换为结构化数据（如JSON、列表、自定义对象），方便后续处理。

3.2.2 核心使用示例

1. 基础提示词模板

Code block

```
from langchain.prompts import PromptTemplate
from langchain_openai import OpenAI

# 定义提示词模板（包含变量：topic、length）
prompt_template = PromptTemplate(
    input_variables=["topic", "length"],
    template="请写一篇关于{topic}的文章，字数控制在{length}字左右，语言简洁明了。"
)

# 填充变量生成具体提示词
prompt = prompt_template.format(topic="LangChain 1.0 组件化开发", length="500")
print("生成的提示词：", prompt)

# 调用LLM生成内容
llm = OpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo-instruct")
response = llm.invoke(prompt)
print("LLM输出：", response)
```

2. 聊天提示词模板

Code block

```
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain.schema import SystemMessagePromptTemplate, HumanMessagePromptTemplate

# 定义系统提示词模板和用户提示词模板
system_template = SystemMessagePromptTemplate.from_template("你是一个{role}，擅长{skill}")
```

```
human_template = HumanMessagePromptTemplate.from_template("请解释{concept}的核心原理")

# 组合为聊天提示词模板
chat_prompt_template = ChatPromptTemplate.from_messages([system_template, human_template])

# 填充变量生成消息列表
chat_prompt = chat_prompt_template.format_prompt(
    role="技术讲师",
    skill="讲解LLM开发框架",
    concept="LangChain 1.0 的Chain组件"
).to_messages()

# 调用聊天模型
chat_llm = ChatOpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo")
response = chat_llm.invoke(chat_prompt)
print("聊天模型输出: ", response.content)
```

3. 输出解析器（结构化输出）

Code block

```
from langchain.prompts import PromptTemplate
from langchain_openai import OpenAI
from langchain.output_parsers import StructuredOutputParser, ResponseSchema

# 定义输出格式（响应 schema）
response_schemas = [
    ResponseSchema(name="component_name", description="LangChain组件名称"),
    ResponseSchema(name="core_function", description="组件的核心功能"),
    ResponseSchema(name="usage_scenario", description="组件的适用场景，用列表形式呈现")
]

# 初始化输出解析器
output_parser = StructuredOutputParser.from_response_schemas(response_schemas)
# 获取格式提示词（告诉LLM需要输出的格式）
format_instructions = output_parser.get_format_instructions()

# 定义提示词模板（包含格式要求）
prompt_template = PromptTemplate(
    input_variables=["component"],
    template="请介绍LangChain 1.0中的{component}组件。{format_instructions}",
    partial_variables={"format_instructions": format_instructions}  # 固定格式要求
```

```
)

# 生成提示词并调用LLM
prompt = prompt_template.format(component="Memory")
llm = OpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo-instruct")
response = llm.invoke(prompt)

# 解析LLM输出为结构化数据（字典）
parsed_output = output_parser.parse(response)
print("结构化输出：", parsed_output)
print("组件核心功能：", parsed_output["core_function"])
```

3.3 链（Chains）：串联组件实现复杂逻辑

Chains 组件用于将多个LangChain组件（如PromptTemplate、LLM、OutputParser、Tool等）串联起来，实现多步流程自动化。LangChain 1.0 提供了更丰富的预定义链，并支持自定义链的灵活构建。

3.3.1 链的类型

- **基础链（LLMChain）**：最基础的链，串联PromptTemplate和LLM，实现“提示词生成→LLM调用”的简单流程。
- **顺序链（SequentialChain）**：按顺序执行多个子链，前一个链的输出作为后一个链的输入，适用于多步推理场景。
- **分支链（RouterChain）**：根据输入内容动态选择不同的子链执行，适用于多场景适配需求（如不同类型的问题对应不同的处理逻辑）。
- **工具链（ToolChain）**：串联LLM和工具，实现“调用工具获取数据→LLM处理数据→生成结果”的流程。

3.3.2 核心使用示例

1. 基础链（LLMChain）

Code block

```
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain_openai import OpenAI

# 定义提示词模板
prompt_template = PromptTemplate(
    input_variables=["question"],
    template="请用简洁的语言回答以下问题：{question}"
)

```

```
# 初始化LLM
llm = OpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo-instruct")

# 构建LLMChain（串联PromptTemplate和LLM）
llm_chain = LLMChain(llm=llm, prompt=prompt_template)

# 执行链
response = llm_chain.invoke({"question": "LangChain 1.0 中Chain组件的作用是什么？"})
print("链的输出：", response["text"])
```

2. 顺序链（SequentialChain）

Code block

```
from langchain.chains import SequentialChain, LLMChain
from langchain.prompts import PromptTemplate
from langchain_openai import OpenAI

# 初始化LLM
llm = OpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo-instruct")

# 定义第一个链：生成文章大纲（输入：topic，输出：outline）
prompt1 = PromptTemplate(
    input_variables=["topic"],
    template="请为{topic}生成一个详细的文章大纲，分5-8个小节。"
)
chain1 = LLMChain(llm=llm, prompt=prompt1, output_key="outline")

# 定义第二个链：根据大纲生成文章（输入：outline，输出：article）
prompt2 = PromptTemplate(
    input_variables=["outline"],
    template="请根据以下大纲撰写一篇完整的文章，语言流畅，逻辑清晰：\n{outline}"
)
chain2 = LLMChain(llm=llm, prompt=prompt2, output_key="article")

# 构建顺序链（串联chain1和chain2）
sequential_chain = SequentialChain(
    chains=[chain1, chain2],
    input_variables=["topic"],
    output_variables=["outline", "article"]  # 输出两个链的结果
)

# 执行顺序链
response = sequential_chain.invoke({"topic": "LangChain 1.0 进阶开发技巧"})
```

```
print("文章大纲: ", response["outline"])
print("\n完整文章: ", response["article"])
```

3. 自定义链（继承Chain类）

Code block

```
from langchain.chains import Chain
from langchain_openai import OpenAI
from pydantic import BaseModel, Field
from typing import Any, Dict

# 定义自定义链的输入参数模型（用于参数校验）
class CustomChainInput(BaseModel):
    question: str = Field(description="用户的问题")
    language: str = Field(default="中文", description="输出语言")

# 自定义链
class MyCustomChain(Chain):
    llm: OpenAI  # 依赖的LLM组件
    input_key: str = "question"  # 输入参数名
    language_key: str = "language"  # 语言参数名
    output_key: str = "answer"  # 输出参数名

    @property
    def input_keys(self) -> list:
        # 定义输入参数列表
        return [self.input_key, self.language_key]

    @property
    def output_keys(self) -> list:
        # 定义输出参数列表
        return [self.output_key]

    def _call(self, inputs: Dict[str, Any]) -> Dict[str, Any]:
        # 核心执行逻辑
        question = inputs[self.input_key]
        language = inputs[self.language_key]

        # 构建提示词
        prompt = f"请用{language}回答以下问题: {question}"

        # 调用LLM
        response = self.llm.invoke(prompt)

        # 返回结果
        return {self.output_key: response}
```

```

# 初始化自定义链
custom_chain = MyCustomChain(
    llm=OpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo-instruct")
)

# 执行自定义链
response = custom_chain.invoke({
    "question": "LangChain 1.0 如何实现组件的灵活组合？",
    "language": "英文"
})
print("自定义链输出：", response["answer"])
```

3.4 记忆（Memory）：管理上下文状态

Memory 组件用于存储和管理对话历史、任务状态等上下文信息，让LLM能够基于历史交互进行连续推理（如多轮对话）。LangChain 1.0 提供了多种记忆类型，适配不同的存储需求和场景。

3.4.1 记忆的类型

- **对话缓冲记忆（ConversationBufferMemory）**：简单存储完整的对话历史，适用于短对话场景。
- **对话缓冲窗口记忆（ConversationBufferWindowMemory）**：只存储最近N轮对话历史，避免历史信息过多导致提示词过长。
- **对话摘要记忆（ConversationSummaryMemory）**：将对话历史总结为摘要存储，节省提示词空间，适用于长对话场景。
- **实体记忆（ConversationEntityMemory）**：提取对话中的实体（如人名、地名、概念）及其相关信息存储，强化实体级别的上下文理解。
- **向量存储记忆（VectorStoreRetrieverMemory）**：将对话历史转换为向量存储在向量数据库中，通过语义检索获取相关历史信息，适用于海量对话历史场景。

3.4.2 核心使用示例

1. 对话缓冲记忆（基础多轮对话）

Code block

```
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
from langchain_openai import ChatOpenAI

# 初始化聊天模型
```

```
chat_llm = ChatOpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo")

# 初始化对话缓冲记忆
memory = ConversationBufferMemory()

# 构建对话链（集成记忆组件）
conversation_chain = ConversationChain(
    llm=chat_llm,
    memory=memory,
    verbose=True  # 开启详细日志，可查看提示词中的对话历史
)

# 多轮对话交互
response1 = conversation_chain.invoke({"input": "介绍一下LangChain 1.0的核心组件"})
print("第一轮回复：", response1["response"])

response2 = conversation_chain.invoke({"input": "这些组件中，哪个最适合用于多轮对话场景？"})
print("第二轮回复：", response2["response"])

response3 = conversation_chain.invoke({"input": "刚才提到的那个组件，具体怎么使用？"})
print("第三轮回复：", response3["response"])

# 查看记忆中的对话历史
print("\n记忆中的对话历史：")
print(memory.load_memory_variables({}))
```

2. 对话缓冲窗口记忆（限制历史轮数）

Code block

```
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferWindowMemory
from langchain_openai import ChatOpenAI

# 初始化窗口记忆（只保留最近2轮对话）
memory = ConversationBufferWindowMemory(k=2)  # k：保留的历史轮数

# 构建对话链
conversation_chain = ConversationChain(
    llm=ChatOpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo"),
    memory=memory,
    verbose=True
```

```
)

# 多轮对话
print("第一轮：")
print(conversation_chain.invoke({"input": "我想学习LangChain 1.0，应该从哪里开始？"})["response"])

print("\n第二轮：")
print(conversation_chain.invoke({"input": "基础组件有哪些需要重点掌握？"})["response"])

print("\n第三轮：")
print(conversation_chain.invoke({"input": "刚才提到的基础组件，具体怎么组合使用？"})["response"])

print("\n第四轮（此时只保留2-3轮历史）：")
print(conversation_chain.invoke({"input": "我刚才问的第一个问题是什么？"})["response"])  # 因记忆窗口限制，无法回忆起第一个问题

# 查看当前记忆中的历史
print("\n当前记忆中的对话历史：")
print(memory.load_memory_variables({}))
```

3.5 工具（Tools）：扩展LLM的外部能力

Tools 组件用于集成外部工具和服务，让LLM能够主动调用这些工具获取信息、执行操作（如搜索、查询数据库、运行代码等），突破LLM自身的知识截止日期和功能局限。LangChain 1.0 提供了丰富的预定义工具，并支持自定义工具。

3.5.1 常用预定义工具

- **搜索引擎工具**：Google Search、Bing Search、DuckDuckGo Search（获取实时信息）。
- **数据库工具**：SQLDatabaseToolkit（连接MySQL、PostgreSQL等数据库，执行SQL查询）。
- **代码执行工具**：PythonREPLTool（执行Python代码，用于数据分析、计算等）。
- **文档处理工具**：PyPDFLoader、Docx2txtLoader（解析PDF、Word文档）。
- **第三方API工具**：WeatherAPI（查询天气）、GitHubAPI（操作GitHub仓库）、SlackAPI（发送Slack消息）。

3.5.2 核心使用示例

1. 调用Python代码执行工具（数据分析示例）

Code block

```python
from langchain.chains import LLMMathChain
from langchain.tools import PythonREPLTool
from langchain_openai import ChatOpenAI
from langchain.agents import create_react_agent, AgentExecutor, load_tools

# 初始化聊天模型
llm = ChatOpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo", temperature=0)

# 加载Python REPL工具
tools = [PythonREPLTool()]

# 构建Agent（用于决定何时调用工具）
agent = create_react_agent(llm, tools, verbose=True)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# 执行需要代码计算的任务
task = "计算1+2*3-4/2的结果，然后生成一个包含该结果的简单柱状图（使用matplotlib）"
response = agent_executor.invoke({"input": task})
print("任务结果：", response["output"])
```

2. 自定义工具

Code block

```python
from langchain.tools import BaseTool, StructuredTool, tool
from langchain_openai import ChatOpenAI
from langchain.agents import create_react_agent, AgentExecutor
from pydantic import BaseModel, Field

# 方式1：使用装饰器定义简单工具
@tool
def add_numbers(a: float, b: float) -> float:
    """
    用于计算两个数字的和。
    参数：
        a: 第一个数字
        b: 第二个数字
    返回：
        两个数字的和
    """
    return a + b

# 方式2：通过类定义复杂工具（支持参数校验）
class SubtractNumbersInput(BaseModel):
    a: float = Field(description="被减数")
    b: float = Field(description="减数")
```

```python

class SubtractNumbersTool(BaseTool):
    name = "subtract_numbers"
    description = "用于计算两个数字的差（a - b）"
    args_schema: type[BaseModel] = SubtractNumbersInput  # 输入参数模型

    def _run(self, a: float, b: float) -> float:
        # 工具核心执行逻辑
        return a - b

# 初始化工具列表
tools = [add_numbers, SubtractNumbersTool()]

# 初始化LLM和Agent
llm = ChatOpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo", temperature=0)
agent = create_react_agent(llm, tools, verbose=True)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# 执行任务（调用自定义工具）
task = "计算 10.5 + 3.2 的结果，再用该结果减去 5.7"
response = agent_executor.invoke({"input": task})
print("最终结果：", response["output"])
```

3.6 代理（Agents）：智能决策与工具调用

Agents 组件是LangChain中实现“智能决策”的核心，能够让LLM根据用户需求，自主决定：是否需要调用工具、调用哪个工具、如何使用工具的输出继续推理，直到完成任务。LangChain 1.0 提供了多种Agent类型，适配不同的决策逻辑和场景。

3.6.1 Agent的核心原理

1. 接收用户输入，分析任务目标；
2. 基于当前上下文和工具列表，决定下一步行动（调用工具/直接生成答案）；
3. 若调用工具，执行工具并获取输出结果；
4. 分析工具输出，判断是否需要继续调用工具（多步工具调用），或直接生成最终答案；
5. 将结果返回给用户。

3.6.2 Agent的类型

- **ReAct Agent**：基于“思考-行动-观察”（Think-Act-Observe）循环的Agent，决策逻辑清晰，适用于大多数通用场景。

- **Zero-Shot ReAct Agent**：无需训练，直接通过提示词让LLM理解决策逻辑，适用于快速适配新工具和新场景。
- **Conversational Agent**：专门用于多轮对话场景，集成了Memory组件，能够记住历史对话信息。
- **Plan-and-Execute Agent**：先规划完成任务的步骤，再逐步执行每个步骤（调用对应工具），适用于复杂的多步骤任务。

3.6.3 核心使用示例

1. ReAct Agent（通用工具调用）

Code block

```
from langchain.agents import create_react_agent, AgentExecutor, load_tools
from langchain_openai import ChatOpenAI

# 初始化LLM（ReAct Agent推荐使用Chat Model）
llm = ChatOpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo", temperature=0)

# 加载工具（此处加载serpapi（搜索引擎）和llm-math（数学计算）工具）
# 需要先设置SERPAPI_API_KEY环境变量（注册https://serpapi.com/获取）
tools = load_tools(["serpapi", "llm-math"], llm=llm)

# 构建ReAct Agent
agent = create_react_agent(llm, tools, verbose=True)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True, handle_parsing_errors="返回错误信息并继续")

# 执行需要多工具协作的任务
task = "查询当前（2025年）全球GDP排名前三的国家，然后计算这三个国家GDP的总和（单位：亿美元）"
response = agent_executor.invoke({"input": task})
print("任务结果：", response["output"])
```

2. 对话Agent（多轮对话+工具调用）

Code block

```
from langchain.agents import create_conversational_agent, AgentExecutor, load_tools
from langchain.memory import ConversationBufferMemory
from langchain_openai import ChatOpenAI

# 初始化LLM
llm = ChatOpenAI(api_key=os.getenv("OPENAI_API_KEY"), model_name="gpt-3.5-turbo", temperature=0)
```

```python

# 加载工具（搜索引擎+数学计算）
tools = load_tools(["serpapi", "llm-math"], llm=llm)

# 初始化对话记忆（存储历史对话）
memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)

# 构建对话Agent
agent = create_conversational_agent(llm, tools, memory=memory, verbose=True)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True, handle_parsing_errors="返回错误信息并继续")

# 多轮对话交互
print("第一轮对话：")
response1 = agent_executor.invoke({"input": "查询2025年中国的GDP预计是多少亿美元？"})
print("Agent回复：", response1["output"])

print("\n第二轮对话：")
response2 = agent_executor.invoke({"input": "用这个数值乘以0.1，得到的结果是多少？"})  # 依赖上一轮的GDP数据
print("Agent回复：", response2["output"])

print("\n第三轮对话：")
response3 = agent_executor.invoke({"input": "对比一下这个结果和2024年印度的GDP，哪个更大？"})  # 依赖前两轮的信息
print("Agent回复：", response3["output"])
```

四、LangChain 1.0 实践案例：构建文档问答系统

本案例将综合运用LangChain 1.0的核心组件（文档加载、文本分割、嵌入模型、向量数据库、链、LLM），构建一个能够处理PDF文档的智能问答系统。用户可上传PDF文档，然后针对文档内容进行提问，系统将基于文档内容生成准确回答。

4.1 案例需求

- 支持加载PDF文档并解析文本内容；
- 将文档文本分割为合适的片段（Chunk）；
- 使用嵌入模型将文本片段转换为向量，存储到向量数据库；
- 用户提问时，通过语义检索从向量数据库中找到相关文本片段；
- 结合相关文本片段和用户问题，调用LLM生成准确回答。

4.2 技术选型

- 文档加载：PyPDFLoader（解析PDF文档）；
- 文本分割：RecursiveCharacterTextSplitter（递归文本分割，适配不同格式文本）；
- 嵌入模型：OpenAIEmbeddings（text-embedding-3-small）；
- 向量数据库：Chroma（轻量级本地向量数据库，无需额外部署）；
- LLM：ChatOpenAI（gpt-3.5-turbo）；
- 链：RetrievalQA（检索+问答链，串联检索和LLM问答）。

4.3 代码实现

Code block

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQA
import os

# 1. 配置环境变量
os.environ["OPENAI_API_KEY"] = "your-openai-api-key"  # 替换为你的API密钥

# 2. 加载PDF文档
def load_pdf_document(pdf_path):
    """加载PDF文档并返回文档对象列表"""
    loader = PyPDFLoader(pdf_path)
    documents = loader.load()  # 加载PDF的每一页作为一个文档对象
    print(f"成功加载PDF文档，共{len(documents)}页")
    return documents

# 3. 文本分割（将长文本分割为短片段，适配嵌入模型的token限制）
def split_text(documents):
    """将文档文本分割为片段"""
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,  # 每个片段的最大token数（近似）
        chunk_overlap=200,  # 片段之间的重叠token数（保证上下文连贯性）
        length_function=len  # 长度计算函数（默认按字符数，可替换为token计数）
    )
    split_docs = text_splitter.split_documents(documents)
    print(f"文本分割完成，共生成{len(split_docs)}个片段")
    return split_docs

# 4. 构建向量数据库（存储文本片段的嵌入向量）
```

```python
def build_vector_db(split_docs):
    """基于分割后的文本片段构建Chroma向量数据库"""
    # 初始化嵌入模型
    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

    # 构建并持久化向量数据库（persist_directory：本地存储路径）
    vector_db = Chroma.from_documents(
        documents=split_docs,
        embedding=embeddings,
        persist_directory="./chroma_db"  # 向量数据库存储在当前目录的chroma_db文件夹
    )
    vector_db.persist()  # 持久化数据库
    print("向量数据库构建完成并持久化")
    return vector_db

# 5. 构建检索问答链
def build_retrieval_qa_chain(vector_db):
    """构建检索+问答链"""
    # 初始化LLM
    llm = ChatOpenAI(
        model_name="gpt-3.5-turbo",
        temperature=0.3  # 降低随机性，保证回答准确性
    )

    # 构建RetrievalQA链（retriever：向量数据库的检索器，k：每次检索返回的相关片段数）
    retrieval_qa_chain = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type="stuff",  # 将所有相关片段拼接进提示词（适合短文档）
        retriever=vector_db.as_retriever(search_kwargs={"k": 3}),
        return_source_documents=True  # 返回回答对应的原始文档片段（用于溯源）
    )
    print("检索问答链构建完成")
    return retrieval_qa_chain

# 6. 问答交互
def qa_interaction(retrieval_qa_chain):
    """与用户进行问答交互"""
    print("\n文档问答系统已启动，输入'quit'退出")
    while True:
        user_question = input("\n请输入你的问题：")
        if user_question.lower() == "quit":
            print("退出系统，再见！")
            break

        # 执行问答链
        result = retrieval_qa_chain.invoke({"query": user_question})
```

```python

        # 输出结果
        print("回答: ", result["result"])

        # 输出相关文档片段（溯源）
        print("\n相关文档片段: ")
        for i, doc in enumerate(result["source_documents"], 1):
            print(f"\n片段{i}（页码: {doc.metadata.get('page', '未知')}）: ")
            print(doc.page_content[:500] + "..." if len(doc.page_content) > 500 else doc.page_content)
```

4.4 案例优化方向

- **文本分割优化**：使用TokenSplitter（基于token计数）替换字符计数，更精准适配嵌入模型的token限制；
- **链类型优化**：对于长文档，使用"map_reduce"或"refine"链类型，避免提示词过长（"stuff"链适合短文档）；
- **向量数据库优化**：使用Pinecone/Weaviate等分布式向量数据库，支持海量文档存储和高并发检索；
- **多文档支持**：扩展代码支持批量加载PDF/Word/网页等多种格式文档；
- **界面优化**：使用Streamlit/FastAPI构建Web界面，提升用户体验。

五、LangChain 1.0 进阶技巧与最佳实践

5.1 性能优化

- **模型选择优化**：根据任务复杂度选择合适的模型（简单任务用gpt-3.5-turbo，复杂任务用gpt-4-turbo），平衡性能和成本；
- **提示词优化**：使用结构化提示词（如明确输出格式、限定思考步骤），减少LLM的无效输出；避免提示词过长，通过摘要、筛选等方式精简上下文；
- **缓存机制**：使用LangChain的Cache组件（如InMemoryCache、RedisCache）缓存LLM的重复调用结果，减少API调用次数和延迟；
- **并行处理**：对于批量任务（如批量文档嵌入、批量问答），使用LangChain的并行链（ParallelChain）或多线程处理，提升效率。

5.2 安全性优化

- **API密钥安全**：避免在代码中硬编码API密钥，使用环境变量、配置文件（加密）或密钥管理服务（如AWS Secrets Manager）存储；
- **输入校验**：对用户输入进行校验和过滤，避免恶意输入（如注入攻击、Prompt Injection）；

- **输出过滤**：使用LangChain的OutputParser或第三方内容审核工具（如OpenAI的Moderation API）过滤LLM的有害输出；
- **工具权限控制**：限制Agent调用工具的权限（如数据库只读权限、API有限访问权限），避免恶意操作。

5.3 可扩展性设计

- **模块化开发**：将不同功能封装为独立的组件（如自定义Chain、自定义Tool），便于复用和维护；
- **配置化管理**：将模型参数、工具配置、链参数等通过配置文件管理，支持动态调整，无需修改代码；
- **日志与监控**：集成LangChain的日志组件（如LLMCallbackHandler）记录LLM调用、链执行、工具调用等日志；使用Prometheus/Grafana等工具监控系统性能和运行状态；